

Generative AI: Assignment 1

Total Marks: 100 | Due Date: 14th September, 9PM

Part 1: Topic Detection and Summarization of News Articles (45 marks)

Problem Statement:

You are given a dataset of news articles (i.e., the BBC News Archive) consisting of about 2,225 articles across 5 categories. Your task is to build a LangChain-powered solution to analyze these articles and perform the following:

1. **Topic Classification:** Determine the main topic/category of each news article. Categorize each article under one of the defined news topics (for example: Business, Entertainment, Politics, Sport, or Tech).
2. **Summarization:** Generate a concise summary of each article, capturing the key points in a few sentences.
3. **Key Entity Extraction:** Identify important entities mentioned in the article, such as notable people, organizations, or locations, and list them.

By the end, your solution should produce a structured overview of the articles, including their detected topic, summary, and any key entities.

Steps to Solve:

Step 1: Load the Dataset

- Use the provided news articles dataset (the *BBC News Full-Text* dataset). This dataset contains news texts along with their categories.
- **Limit the data** to the first 30 articles for this assignment (to manage processing time). For instance, if the data is in a DataFrame `df`, use `df.head(30)` to select the first 30 records.
- *Dataset reference:* BBC News Archive (Kaggle) – a collection of 2,225 BBC news articles categorized into 5 topics (Business, Entertainment, Politics, Sport, Tech).
- **Expected Output:** Loaded in dataframe.

Step 2: Define the Topic Classification Task (10 marks)

- Use LangChain to create a prompt or chain that classifies the topic of each article.
- Formulate a prompt template that guides the model, for example:
“Analyze the following news article and identify its topic as one of the following categories: Business, Entertainment, Politics, Sport, or Tech.”
- Ensure the model’s output is a single category label. You may use few-shot examples in the prompt (i.e., provide a couple of article snippets with their correct category) to improve accuracy.
- **Expected Output:** Show this works for a sample datapoint.

Step 3: Define the Summarization Task (10 marks)

- Create a LangChain pipeline or prompt for summarizing each article.
- For instance, you might use a prompt like:
“Summarize the main points of the following news article in 2-3 sentences.”
- Ensure the summary captures the essence of the article (who/what/when/where/why, as applicable) without personal commentary.
- **Expected Output:** Show this works for a sample datapoint.

Step 4: Key Entity Extraction (10 marks)

- When extracting entities, use the LLM to find key names or terms in the article. You can chain this after summarization or as a separate step.
- Example prompt: “From the article above, list the names of any important people, organizations, or places mentioned.”
- The model should return lists or a comma-separated string of entities for each category (you can decide how to structure this output).
- **Expected Output:** Show this works for a sample datapoint.

Step 5: Update the DataFrame with Results (15 marks)

- Iterate over each article (each row of your DataFrame) and apply the classification, summarization and entity extraction prompts. Collect the results for each article.
- Add new columns to the DataFrame for the outputs, for example: `Detected_Topic`, `Summary`, and `Key_Entities`.
- Populate these columns with the model’s outputs for each corresponding article.
- **Expected Output:** Get a Pandas dataframe with all required additional columns. Then, get the final Pandas dataframe with all original and new columns together.

Example Output in JSON (should be a merged Pandas dataframe finally):

```
{
  "Article_ID": 42,
  "Title": "New Tech Innovation in Silicon Valley",
  "Article_Text": "A breakthrough technology unveiled today is set to transform the industry... [excerpt]",
  "Detected_Topic": "Tech",
  "Summary": "A tech company in Silicon Valley announced a breakthrough innovation that is expected to transform the industry. The article describes how this new technology could impact consumers and businesses in the coming years.",
  "Key_Entities": ["Silicon Valley", "tech company", "new technology"]
}
```

Part 2: Job Postings Analysis – Role Categorization and Requirements Extraction (55 marks)

Problem Statement:

You have a dataset of job postings, each with a **job title** and a detailed **job description**. This dataset was compiled by scraping listings from various job portals (e.g., Glassdoor, Indeed, etc.). The goal is to use LangChain and an LLM to automatically analyze each job posting and extract structured information:

1. **Job Category Classification:** Determine a broad category or domain for the job. For example, classify the job role into categories such as *Technology/IT*, *Finance*, *Marketing*, *Healthcare*, etc., based on the description.
2. **Key Requirements Extraction:** Extract important requirements and qualifications from the job description. This includes:
 - **Required Skills/Technologies:** Identify specific skills, programming languages, tools, or domain knowledge mentioned (e.g., Python, project management, CRM software).
 - **Education Level:** Identify if a certain education level is required or preferred (e.g., Bachelor's degree, MBA, etc.).
 - **Experience:** Identify any mention of required experience, such as years of experience or level (e.g., "3+ years experience in...", or phrases like "senior-level experience").

The final output should augment each job posting with a category label and several fields detailing the extracted requirements (skills, education, experience, etc.), providing a structured summary of the job listing.

Steps to Solve:

Step 1: Load the Dataset

- Use the provided job postings dataset (for example, a CSV with columns like `Job_Title` and `Job_Description`). This dataset contains a variety of roles across different industries, scraped from multiple source.
- Load the data into a `DataFrame`. Each row represents one job posting.
- **Limit the data** to the first 25 job postings to keep the processing feasible. (For instance, use `df.head(25)` after loading.)
- *Dataset reference:* Job Title and Job Description Dataset (Kaggle) – a collection of job titles with descriptions scraped from online job boards.
- **Expected Output:** Loaded in dataframe.

Step 2: Define the Job Category Classification Task (10 marks)

- Develop a prompt to classify the job posting into a broad category. This might rely mostly on the `Job_Title` (and possibly the description).

- For example: “Given the following job title and description, categorize the job into one of the following domains: Technology/IT, Finance, Marketing, Healthcare, Education, Others. Job: <Job Title>. Description: <Job Description>. Domain category:”
- You may adjust or expand the list of categories based on the dataset’s content. If unsure, use a generic “Other” category as a fallback.
- The LLM should return a single category label. Using a few examples in the prompt (job title + snippet -> category) can help guide the model.
- **Expected Output:** Show this works for a sample datapoint.

Step 3: Define the Requirements Extraction Task (30 marks)

- Now create prompts (or a single composite prompt) to extract the key requirements from the job description: skills, education, and experience. You can handle this in one go or separately for each sub-task. For clarity, you might do them one by one:
 - **Skills:** “List the key skills or technologies mentioned in the following job description.”
 - **Education:** “What is the minimum education level required or preferred for this job (if stated)?”
 - **Experience:** “What is the minimum years of experience or experience level required for this job (if mentioned)?”
- Alternatively, use a single prompt: “Extract the required skills, education level, and years of experience from the job description below.” Ensure the output is structured (you could ask it to respond in JSON with fields for Skills, Education, Experience).
- **Expected Output:** Show this works for a sample datapoint.

Step 4: Apply the LLM Chain to Each Job Posting (10 marks)

- Loop through each job posting in the DataFrame. For each entry, do the following:
 1. Feed the job title/description into the classification prompt to get the `Job_Category`.
 2. Feed the description into the extraction prompts to get `Required_Skills`, `Education_Required`, and `Experience_Required`. (If using separate prompts, run them sequentially; if combined into one prompt that outputs all, run that once per posting.)
- Make sure to handle cases where a particular piece of information is not mentioned. For example, if no education requirement is stated, the model (or you in post-processing) might output “Not specified” or `null` for that field.
- **Expected Output:** Get a Pandas dataframe with all required additional columns.

Step 5: Update the DataFrame with New Columns (5 marks)

- Add new columns to the DataFrame for each of the outputs: e.g., `Predicted_Category`, `Required_Skills`, `Education_Required`, `Experience_Required`.
- As you retrieve the LLM’s output for each posting, fill these columns accordingly. `Required_Skills` can be a single string (comma-separated skills) or a Python list of skills; ensure consistency.
- After processing all 25 postings, verify the outputs for correctness. You might spot-check a few to see if the extracted info aligns with the description.
- **Expected Output:** Final Pandas dataframe with all original and new columns together.

Example Output in JSON (should be a merged Pandas dataframe finally):

```
{  
  "Job_Title": "Senior Data Analyst",  
  "Job_Description": "We are seeking a Senior Data Analyst to join our team...  
Responsibilities include analyzing sales data, creating dashboards, and reporting  
insights. **Requirements:** 5+ years experience in data analysis, proficiency in  
SQL and Python, familiarity with BI tools. Bachelor's degree in Finance,  
Statistics, or related field required. Excellent communication skills and  
attention to detail are a must.",  
  "Predicted_Category": "Finance/Analytics",  
  "Required_Skills": ["SQL", "Python", "Business Intelligence tools", "data  
analysis"],  
  "Education_Required": "Bachelor's degree (Finance, Statistics, or related)",  
  "Experience_Required": "5+ years"  
}
```

In this example, the assignment system categorized the job as *Finance/Analytics* based on the title/description, and extracted key skills (SQL, Python, BI tools, etc.), the education requirement (Bachelor's degree in a related field), and the experience requirement (5+ years). Each of these pieces was obtained via the LangChain prompts and added to the structured output.

Bonus (optional) – 20 marks

If you can complete the analysis for **ALL** the rows in the dataset, you'll get Bonus marks of 10 marks. This applies to Part 1 and Part 2 separately. Thus total maximum Bonus available is 20 marks.

Important Note: You might hit rate limits from Groq if trying to do for the entire dataset. Thus, it is advised to use Ollama instead and leverage SLMs (small language model) of your choice. Make sure to balance inference speed vs. accuracy for each use case.
